

CAMPUSLINK / STUDY EDITION

Know your technology. Explain your project.

Technology stack, core concepts, feature walkthroughs, and interview preparation.

A practical guide based on the code in your CampusLink project. Covers the student, recruiter, and institute workspaces, voice communication practice, student record editing, and contest achievements.

READ / UNDERSTAND / PRACTICE

Prepared 6 October 2026. Versions refer to the project lockfile, not the newest available releases.

Contents

Topics 01-23	p.	Topics 24-46	p.
01 / How to explain CampusLink	3	24 / Supabase and Google sign-in	26
02 / Stack at a glance	4	25 / File uploads, Multer and PDF parsing	27
03 / Runtime dependency inventory	5	26 / Email, the outbox and reminders	28
04 / Development dependency inventory	6	27 / Recruitment workflow and state machines	29
05 / Architecture and source map	7	28 / Assessments, grading and verified skills	30
06 / HTML, CSS, Tailwind and accessibility	8	29 / Readiness, eligibility and hybrid matching	31
07 / JavaScript concepts behind the app	9	30 / NLP, TF-IDF and cosine similarity	32
08 / TypeScript and type safety	10	31 / Logistic regression and ML evaluation	33
09 / React, JSX and components	11	32 / Optional OpenAI coaching and external ML	34
10 / Hooks, lifecycle and cleanup	12	33 / Voice communication practice	35
11 / Next.js routing and rendering	13	34 / Contest points, streaks and badges	36
12 / TanStack Query and server state	14	35 / Tests, linting, formatting and builds	37
13 / Zustand, sessions and browser persistence	15	36 / Deployment, environment variables and Git	38
14 / React Hook Form, Zod and validation	16	37 / Feature walkthroughs from screen to storage	39
15 / Charts, icons, sliders, fonts and motion	17	38 / Troubleshooting and design tradeoffs	40
16 / Node.js, npm and the event loop	18	39 / Interview questions: frontend	41
17 / Express and middleware	19	40 / Interview questions: backend and data	42
18 / HTTP, Axios, JSON and service adapters	20	41 / Interview questions: AI, voice and achievements	43
19 / Password authentication and sessions	21	42 / Glossary: application and web concepts	44
20 / Authorization, RBAC and institute ownership	22	43 / Glossary: security, data and intelligence	45
21 / Web security concepts in this project	23	44 / Revision plan and project demonstration	46
22 / SQLite, PostgreSQL and the repository layer	24	45 / Source evidence and further study	47
23 / Transactions, concurrency and audit trails	25	46 / Official references for further study	48

Use the PDF bookmarks or click a topic to jump directly to it.

01 / How to explain CampusLink

CampusLink is a campus placement and preparation platform for students, institute placement teams, and recruiters. It connects verified skills, student profiles, assessments, campus recruitment drives, interviews, offers, documents, and learning activity. The current work also includes voice communication practice, institute student record editing, and contest streaks and badges.

A 30-second project explanation

"CampusLink is a full-stack campus placement platform. I use Next.js, React and TypeScript for the interface, and a Node.js Express API for business operations. Data can persist in SQLite locally or PostgreSQL on the server. The system checks roles and institute ownership, manages the recruitment workflow, and provides preparation feedback using local analysis and optional AI integrations."

A two-minute explanation

Start with the problem: student preparation and campus recruitment information are spread across separate tools. Describe the three roles. Students maintain profiles and practice; institutes coordinate placement activities and manage their own students; recruiters request campus drives and review authorized applicants. Then explain how a request travels from a component to a service, the API, permission checks, and the database. Finish with a concrete feature such as a contest award or an institute profile update.

Four words you should be able to define

- **Frontend:** the screens and browser interactions a person uses.
- **Backend:** the server that validates requests and executes protected operations.
- **Database:** persistent records that remain available after restarting an application.
- **Integration:** a connection to an external provider, such as storage, email or optional AI coaching.

Use this guide accurately

This is a repository-based study guide, not a complete textbook for every dependency. The source code is the primary evidence for project behavior. Official documentation is linked for further study. Hosted providers require configuration and credentials; their presence in the code does not prove that a live deployment is connected. Older frontend-only paragraphs in README.md describe an earlier stage and should not override the current server implementation.

What to practice first

Learn the project explanation, the architecture, the three kinds of state, and the difference between authentication and authorization. Then trace one feature through its files. Use the question-and-answer pages to practice aloud instead of memorizing library names alone.

02 / Stack at a glance

Layer	Actual technology and responsibility
Language	TypeScript for frontend, domain rules and Express source; JavaScript is the runtime output.
Interface	React 19 and React DOM; reusable components, hooks and interactive screens.
App framework	Next.js 15 App Router, navigation, static assets, frontend builds and API rewrites.
Styling	HTML, CSS, Tailwind CSS 4 through PostCSS, local Inter fonts and existing pastel design tokens.
Data and state	TanStack Query for fetched data; Zustand for session state; React state for local interactions.
Forms	React Hook Form, Zod and @hookform/resolvers. Some small forms use controlled React state directly.
Interface libraries	Framer Motion, Lucide, Recharts, Swiper, and an optional Rive authentication animation.
HTTP	Axios in the frontend; native fetch in backend provider integrations; JSON service calls under /api/v1.
API server	Node.js 24 and Express 5, with middleware, service dispatch, validation and scoped records.
Database	Node's built-in SQLite locally; PostgreSQL through pg; shared repository methods and transactions.
Optional providers	Supabase storage and Google auth, Resend email, OpenAI coaching, and an external ML API adapter.
Local intelligence	Weighted readiness, eligibility rules, TF-IDF matching, text heuristics and logistic regression.
Quality and tooling	Vitest, Supertest, TypeScript checks, ESLint, Prettier, esbuild, npm and Git.
Hosting configuration	Vercel for the frontend and Render for the Express API; production needs external database, storage and email.

Version vocabulary

package.json declares allowed version ranges. A caret such as ^15.5.0 allows compatible updates within that major version under normal semantic-version rules. package-lock.json records the resolved dependency tree. The next two sections show locked versions in this workspace, which can differ from the minimum versions in package.json.

"This project is a Next.js and Express TypeScript application. Its database layer supports SQLite and PostgreSQL. I would not call it MERN: MongoDB is not the application's database."

03 / Runtime dependency inventory

These are direct dependencies recorded in package.json, with versions resolved by package-lock.json. Their role is described below; a configured dependency can still be optional at runtime.

Package	Locked version	Purpose in this repository
@fontsource/inter	5.3.0	Local Inter font files used by the website.
@hookform/resolvers	3.10.0	Connects Zod schemas to React Hook Form.
@rive-app/react-canvas	4.36.0	Optional interactive Rive authentication character.
@tanstack/react-query	5.104.0	Server data, caching, refetching and invalidation.
axios	1.20.0	Shared HTTP client, cookies, timeouts and interceptors.
cookie-parser	1.4.7	Reads request cookies in Express.
cors	2.8.6	Restricts permitted browser origins.
dotenv	18.0.5	Loads server environment configuration.
express	5.2.1	HTTP routing and middleware for the API.
express-rate-limit	8.7.0	Bounds authentication, AI and general request rates.
framer-motion	12.43.0	React animation, scroll motion and reduced-motion support.
helmet	8.3.0	Security-related HTTP response headers.
lucide-react	0.468.0	Consistent SVG interface icons.
multer	2.4.0	Parses multipart file uploads.
next	15.5.27	Routing, application framework, build and API proxy.
pdf-parse	1.1.1	Extracts text from uploaded text-based PDF resumes.
pg	8.23.1	PostgreSQL connection pool and SQL execution.
react	19.3.0	Components, state, hooks and UI composition.
react-dom	19.3.0	Connects React rendering to the browser DOM.
react-hook-form	7.89.0	Form state, validation and submission handling.
recharts	3.10.1	React charts for dashboards and analytics.
swiper	14.3.0	Responsive sliders, carousel keyboard and accessibility modules.
tsx	4.23.15	Development support for TypeScript execution.
zod	3.25.76	Runtime request, form and provider-response validation.
zustand	5.0.15	Small shared session store and persistence.

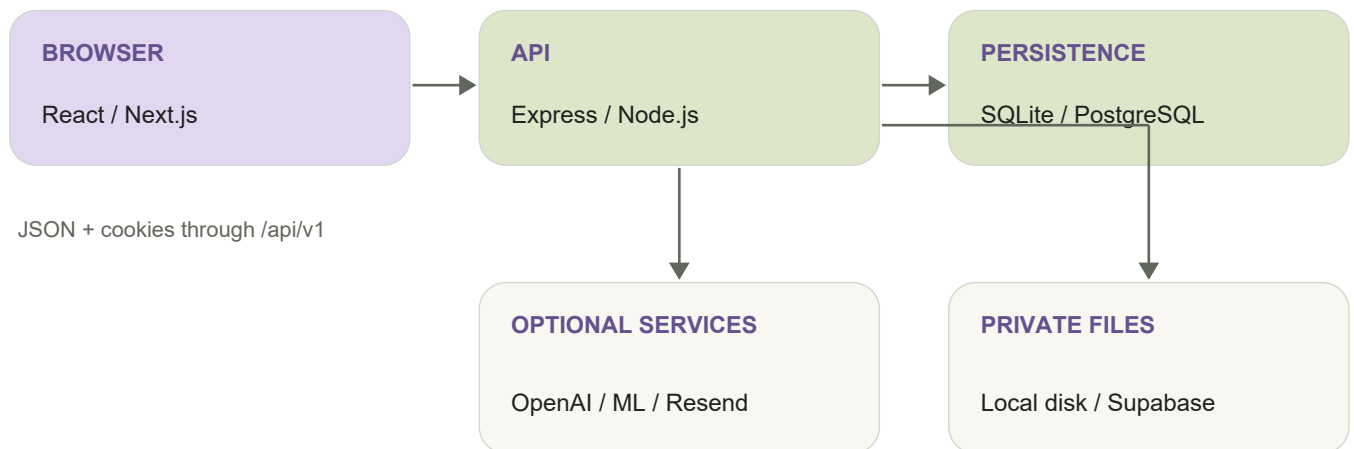
04 / Development dependency inventory

Development dependencies support builds, checking and tests. They are different from services students interact with. @types packages provide compile-time descriptions of JavaScript libraries; they do not run a separate service.

Package	Locked version	Purpose in this repository
@eslint/eslintrc	3.3.7	Compatibility helper for ESLint configuration.
@tailwindcss/postcss	4.3.3	Tailwind integration with the CSS build pipeline.
@types/cookie-parser	1.4.10	Type declarations for the corresponding JavaScript package.
@types/cors	2.8.19	Type declarations for the corresponding JavaScript package.
@types/express	5.0.6	Type declarations for the corresponding JavaScript package.
@types/multer	2.3.0	Type declarations for the corresponding JavaScript package.
@types/node	22.20.5	Type declarations for the corresponding JavaScript package.
@types/pdf-parse	1.1.5	Type declarations for the corresponding JavaScript package.
@types/pg	8.23.1	Type declarations for the corresponding JavaScript package.
@types/react	19.3.0	Type declarations for the corresponding JavaScript package.
@types/react-dom	19.3.0	Type declarations for the corresponding JavaScript package.
@types/supertest	7.2.1	Type declarations for the corresponding JavaScript package.
esbuild	0.28.2	Bundles backend entry points to CommonJS output.
eslint	9.39.5	Source-code linting.
eslint-config-next	15.5.27	Next.js-related lint rules.
prettier	3.9.9	Consistent source formatting.
supertest	7.3.0	Exercises Express endpoints in tests.
tailwindcss	4.3.3	Utility CSS and base styling compilation.
typescript	5.9.3	Static type checks for frontend and backend source.
vitest	4.1.11	Test runner and mocking utilities.

Node.js itself is a runtime requirement, not a package in these tables. package.json specifies Node 24.x. The existing @types/node package is from the 22.x type family; dependency declarations and runtime versions are related but not identical.

05 / Architecture and source map



Demo mode instead uses the browser mock adapter and localStorage.

How the layers cooperate

A feature component renders a screen. A hook or event handler calls a service. In API mode, the service proxy sends a request to Express. The server identifies the account, validates the arguments, checks authorization, executes the domain rule, persists the change, and returns JSON. React Query refreshes the relevant cached data so the visible screen agrees with the saved record.

Source area	What to look for
src/components/platform.tsx	Selects the feature view from the current URL.
src/features and src/components	Screens, reusable UI, dashboard, forms and interaction.
src/hooks / src/store/session.ts	Fetching, voice lifecycle and shared session state.
src/services/platform.domain.ts	Reusable local business behavior and demo operations.
src/services/api and platform.service.ts	Axios client and API-versus-demo adapter.
server/app.ts / services.ts	HTTP endpoints, middleware, service policy and request schemas.
server/workspace.ts / db.ts	Account-scoped workspaces, persistence and transactions.
server/auth.ts / google.ts	Password sessions and optional Google PKCE flow.
src/utils / server/nlp.ts / ml.ts	Eligibility, scoring, achievements, text analysis and models.

Why layers help

The visual design can change without rewriting grading or permission rules. SQLite can be replaced with PostgreSQL through the database adapter without making every component issue SQL. Pure rules can be tested separately from HTTP and browser behavior. Shared domain code still needs server-side validation: browser code is editable by the user.

06 / HTML, CSS, Tailwind and accessibility

What each one does

HTML gives a page structure: headings, forms, labels, buttons, tables and links. CSS controls layout, color, spacing, typography, transitions and responsive behavior. Tailwind supplies a utility CSS system and base styles through the PostCSS pipeline. CampusLink also contains substantial custom stylesheets; it is not exclusively a utility-class implementation.

Concepts used in this project

- **DOM:** the browser's element tree. React ultimately updates this tree.
- **Box model:** content, padding, border and margin determine an element's occupied space.
- **Flexbox:** arranges items along a main axis; useful for toolbars and aligned actions.
- **Grid:** lays out rows and columns; useful for dashboard cards and badge galleries.
- **Media queries:** change layouts for narrower screens and different input devices.
- **CSS variables:** tokens such as `--lavender` and `--sage` keep colors consistent.
- **Cascade and specificity:** determine which competing CSS declaration wins.
- **Transitions:** animate changes between property values over time using an easing curve.

Sidebar example

dashboard.css keeps the desktop sidebar narrow and expands it on hover or keyboard-visible focus. Width and shadow transition for 320 milliseconds. The content reserves the collapsed width, so expansion overlays rather than repeatedly reflows the dashboard. Mobile devices use a separate drawer. Reduced-motion preferences should avoid unnecessary animation.

Accessibility is an implementation concern

Use a real button for an action and a link for navigation. Associate form labels with inputs. Keep keyboard focus visible. A modal needs a meaningful name, Escape handling and a focus trap. The existing Modal component restores focus on close. A hover-only action needs a keyboard and touch alternative. Progress indicators should expose values, and charts need textual context.

"Tailwind helps with the CSS foundation, while custom CSS expresses our theme and the responsive dashboard layout. We also support keyboard navigation and use semantic controls."

Common follow-up

Is responsive design the same as making everything smaller? No. It changes the arrangement: cards can stack, tables can scroll, and navigation can become a drawer. Readable text and usable controls still need enough space.

07 / JavaScript concepts behind the app

TypeScript builds on JavaScript, so these concepts are useful even when most source files end in `.ts` or `.tsx`.

Values, functions and collections

- **const and let:** `const` prevents rebinding a variable; it does not make an object immutable. `let` allows rebinding.
- **Objects and arrays:** objects hold named fields; arrays hold ordered items such as contest history.
- **map:** transforms a collection, for example converting badge rules into badge cards.
- **filter:** selects matching items, for example offers with a particular status.
- **reduce:** combines many values into one, such as a points total or weighted score.
- **Destructuring and spread:** unpack or copy fields. Spreading an object makes a shallow copy; `structuredClone` copies supported nested data more deeply.
- **Optional chaining:** `student.branch?.trim()` avoids accessing a missing value. Nullish coalescing falls back only for null or undefined.

Asynchronous work

A Promise represents a future result or failure. `async` functions return Promises. `await` pauses that function until a Promise settles, while other work can continue. `try/catch` handles failures such as an API rejecting a profile update. `Promise.all` is useful for independent work; dependent writes should preserve their required sequence.

Events and closures

A click or speech-recognition result triggers an event handler. A closure allows a function to access variables from its surrounding scope. Event listeners need cleanup when a screen unmounts; otherwise they may keep running or refer to old state. Timers, observers and microphone-related handles also need lifecycle management.

Modules and errors

Imports and exports divide the program into reusable files. JSON serializes data for transport and storage but does not preserve functions. Custom domain errors describe expected failures; unexpected infrastructure errors should not expose database details to a browser.

"A contest completion updates data asynchronously. I await the save before refreshing the screen, and I handle errors so the interface does not claim that an unsuccessful operation completed."

Important distinction

The browser and Node both execute JavaScript, but their APIs differ. The browser has DOM and speech APIs. Node has filesystem access, server sockets and `node:crypto`. A browser bundle must not contain server credentials or import private server configuration.

08 / TypeScript and type safety

What it is and why it is used

TypeScript adds a static type system to JavaScript. It helps detect incompatible values during development: a Student has a numeric CGPA, a Role is one of the allowed role strings, and a Contest has a defined shape. Types improve editor hints and make refactoring safer, but they are removed from the runtime output.

Concepts to understand

- **Interface:** describes an object's shape; `src/types/index.ts` defines Student, Contest, DemoData and related contracts.
- **Type alias:** names a type, including unions or derived forms.
- **Union:** a value may be one of several alternatives, such as `'student' | 'recruiter' | 'campus'`.
- **Optional property:** `branch?: string` permits a missing branch value.
- **Generics:** describe reusable operations without throwing away information, for example `db.get<DemoData>()`.
- **Partial<T>:** makes each field optional; useful for patches, but too broad by itself for permissions.
- **Pick / Omit:** select or exclude properties from a type to define narrower contracts.
- **Narrowing:** an if statement or schema check gives a more specific safe type.
- **unknown versus any:** unknown requires a check before use; any disables many protections.

A small teaching example

```
type Role = 'student' | 'recruiter' | 'campus';
interface AcademicRecord {
  cgpa: number;
  branch?: string;
}
const record: AcademicRecord = { cgpa: 8.4 };
// cgpa: '8.4' would be a type mismatch.
```

Compile-time and runtime are different

A caller can send an invalid JSON body even when your own frontend compiles. Therefore the API also uses Zod at runtime. Institute record updates use a strict allowed-field schema rather than blindly accepting `Partial<Student>`, which would also describe protected fields such as XP or verified skills.

What the configuration contributes

`tsconfig.json` enables strict checking and the `@/` source-path alias. Separate frontend and server type-check commands catch errors across the layers. TSX combines TypeScript with JSX UI markup. `tsconfig.tsbuildinfo` is an incremental build cache, not source code or a database.

"TypeScript checks our code before it runs. Zod checks the actual values entering the system while it runs. We use both because they solve different problems."

09 / React, JSX and components

The basic idea

React builds an interface from components. A component describes the UI for its current inputs and state. JSX is a JavaScript syntax extension for describing elements; it is not an HTML string. React DOM renders the resulting tree into browser elements.

Project examples

PageHeader, Badge, Modal and Button are reusable interface components. StudentDashboard, PeoplePage, CommunicationPractice and ContestsPage assemble those pieces into feature screens. The same ContestProgress component displays streaks and badges for students and institute staff with different surrounding navigation.

Concepts used repeatedly

- **Props:** data and callbacks passed from a parent, such as student and onSaved.
- **State:** component-owned values that can change, such as the answer or whether an editor is open.
- **Composition:** larger screens are assembled from smaller components instead of repeating markup.
- **Conditional rendering:** show a locked badge, an error, a completed contest or an edit button depending on state.
- **List keys:** stable identifiers help React preserve the identity of items across renders.
- **Controlled input:** the displayed input value comes from state and onChange updates that state.
- **Reconciliation:** React compares successive UI descriptions and updates affected browser elements.
- **One-way data flow:** parents provide inputs; children report changes through callbacks.

Why immutable updates matter

Change data through the intended state setter or adapter, rather than silently mutating an object used by the UI. New references help React and caches detect updates. The project clones workspace data before applying domain mutations and then persists the result.

A common misconception

A rerender is not a full page reload. It means React recalculates a component's UI. It can happen after a local state change or when a query returns new data. Browser navigation and server rendering are separate concerns managed with Next.js.

"I reuse components to keep behavior and styling consistent. For example, the institute student editor uses the same modal and form-field components as other screens, while its save operation goes through a campus-specific service."

10 / Hooks, lifecycle and cleanup

The hooks used in CampusLink

Hook	Responsibility and example
useState	Local input values, selected tab, loading flag and open editor.
useEffect	Synchronizes with external activity, listeners, timers or focus behavior.
useRef	Holds a DOM reference or recognizer handle without causing a rerender when changed.
useCallback	Keeps a callback identity stable when appropriate; the editor close callback is an example.
useSyncExternalStore	Connects to an external subscription; used in hydrated motion preference handling.
Custom hooks	usePlatform encapsulates dashboard fetching; useVoicePractice encapsulates speech lifecycle.

Effect dependencies

An effect's dependency list describes which values can change its behavior. Omitting a required dependency can create a stale closure. Recreating a function on every render can also rerun an effect that depends on that function. Stabilize callbacks where identity is important, rather than adding memoization to everything.

Cleanup example

The voice hook stops recognition, removes callbacks, and clears timers when the component unmounts or a take ends. The modal removes its keyboard listener and restores the earlier focus and body scrolling. A pointer-following character cancels requestAnimationFrame work when it is no longer needed.

Rules of hooks

Call hooks at the top level of a component or another hook. Do not call them conditionally inside a branch or loop: React needs a consistent call order. Custom hooks let related behavior be reused while keeping this order predictable.

Effects versus calculations

Computing badge progress from stored history is a pure calculation, so it does not need an effect that creates more state. Effects are for synchronizing with something outside rendering. Pure calculations are easier to test and avoid two sources of truth.

"I use refs for handles like the microphone recognizer, state for visible UI values, and effects for starting or cleaning up external activity. Derived streak and badge values come directly from completion history."

Debugging question

Why might a timer run twice in development? Development checks can exercise effect setup and cleanup to reveal unsafe lifecycle logic. Correct cleanup matters; merely removing development checks hides the bug rather than addressing it.

11 / Next.js routing and rendering

What Next.js adds to React

Next.js provides an application framework: routing, builds, static assets, server/client boundaries, navigation and deployment support. React supplies the component model; Next.js organizes and delivers the application.

How this repository actually uses it

`src/app/layout.tsx` loads global styling and providers, then renders Platform. The root and catch-all [...slug] page files return no separate feature markup. Platform reads the pathname and chooses a student, recruiter, campus or public screen. This is a heavily client-driven workspace inside the App Router, not a collection of separate server-rendered feature pages.

Concepts to know

- **App Router:** the app directory defines routes and layouts.
- **Layout:** a shared wrapper that can persist across navigation.
- **Client component:** uses 'use client' for state, events or browser hooks.
- **Server component:** rendered in the server-side component system; it cannot directly use browser APIs.
- **Hydration:** React connects browser interactivity to initially delivered markup.
- **Dynamic import:** defers loading a component; the optional `Rive` character is loaded with server rendering disabled.
- **Link and useRouter:** support navigation without manually rebuilding the entire application.
- **public directory:** serves static images, icons and animation assets.
- **Metadata:** the application title and description appear in the root layout.

The API rewrite

`next.config.mjs` proxies `/api/v1` requests to the Express API origin. The browser can use a same-origin path, while the server address is configured separately with `API_INTERNAL_URL`. The rewrite is part of configuration and requires the correct value when building for deployment.

What to avoid claiming

This repository's business API is Express; it is not implemented primarily as Next.js route handlers. It also does not use every Next.js capability simply because Next.js supports it. Explain the actual client-driven routing arrangement if asked how pages are assembled.

"Next.js hosts the frontend and routes users through the workspaces. Express handles protected business operations. The frontend proxies API requests under `/api/v1` so the browser uses a consistent endpoint."

12 / TanStack Query and server state

The problem it solves

Server data is asynchronous and shared. A dashboard can be loading, stale, failed or updated elsewhere. TanStack Query manages fetching, caching, errors and refetching so components do not each build their own ad hoc fetching system.

Concepts in the actual code

- **Query function:** an async operation that returns data, such as `studentService.getDashboard`.
- **Query key:** the identity of cached data. `usePlatform` uses `['platform', user.id]`; student lists include role and account ID to avoid reusing another account's list.
- **Loading and error states:** let the UI represent pending work or a failure honestly.
- **Enabled condition:** postpones a request until an account or selected student exists.
- **Invalidation:** marks cached results stale after a mutation; active queries can then fetch fresh results.
- **setQueryData:** updates a cache immediately when a successful save already returned the updated record.
- **Stale data:** cached data may still be shown while a refresh obtains a newer result.
- **Retries:** some transient failures can be retried; retry behavior should suit the operation.

Institute edit example

The institute saves through `campusService.updateStudent`. On success, `PeoplePage` replaces the affected cached student with the returned profile, invalidates the student list and candidate ranking, and refreshes the workspace. This lets the visible data respond promptly while reconciling with persisted data.

Query cache is not the database

The cache lives in the frontend and can disappear. The server database remains the authority for API-mode records. A student can also change their own record from another browser. An institute must still re-fetch or invalidate cached records when freshness matters.

Three types of state

State category	CampusLink example
Local UI state	Open modal, text in a search box, selected tab.
Shared client state	Session display information in Zustand.
Server state	Student records, documents, contest history and drives cached by Query.

"React Query handles fetched records and their freshness. It avoids putting every API result into a global store. After saving, I update or invalidate the relevant query so the screen reflects the server."

13 / Zustand, sessions and browser persistence

What Zustand does

Zustand is a small shared client-state store. `src/store/session.ts` holds the signed-in user, remember preference, and setter functions. Components select the fields they need without passing the same session props through every level of the tree.

Concepts used here

- **Store:** a shared object with values and actions that update them.
- **Selector:** picks a needed part of the store, such as `state.user`.
- **Persistence middleware:** saves and restores state through a configured storage interface.
- **localStorage:** origin-scoped browser storage that usually survives closing and reopening the browser.
- **sessionStorage:** origin-scoped storage associated with a browser tab session.
- **Hydration of persisted state:** restores saved client information after the application starts.

Remember me is not authorization

The frontend can remember user display information, but it cannot establish trusted permissions from that information. In API mode, the backend authenticates the HTTP-only session cookie and returns the actual server identity through `/auth/me`. Editing browser storage cannot make a student a legitimate campus administrator.

Demo-mode persistence

The mock adapter saves the sample workspace in `localStorage` under `campuslink-data`. Profile updates, contests and practice feedback can persist after reloading in that browser. This is one demonstration dataset, not separate production records for every real institution. API mode instead relies on server account scope and database records.

Logout and stale sessions

Logout revokes the backend session when applicable, clears client session state and clears query data. This matters on shared devices: a later user should not see cached records belonging to the earlier user. Session restoration must also avoid replacing a newly signed-in user with the result of an older in-flight request.

"Zustand keeps lightweight session information available to the interface. The server session cookie is the trusted identity in API mode. Browser persistence is a convenience, not a security boundary."

Tradeoff

A store makes shared UI state convenient, but copying server records into it can create competing sources of truth. Keep authoritative server records in the database and use Query for their frontend cache.

14 / React Hook Form, Zod and validation

How the tools fit together

React Hook Form manages field registration, form state and submission. Zod describes which input values are valid. @hookform/resolvers connects a Zod schema to React Hook Form. Some smaller CampusLink forms, including the institute editor, instead use controlled React inputs and call a shared Zod schema directly.

Validation concepts

- **Schema:** a machine-readable description of valid input.
- **parse:** returns valid data or throws a validation error.
- **safeParse:** returns success/data or failure/error for explicit handling.
- **Constraints:** min/max lengths, numeric bounds, allowed enum values and patterns.
- **Coercion or conversion:** browser number inputs still expose text values; convert deliberately before validation.
- **Strict object:** rejects fields that are not allowed by the schema.
- **Refinement:** adds a custom rule, such as requiring at least one field in an update.
- **Client validation:** helps a person fix input before sending it.
- **Server validation:** protects the operation when any client sends data directly.

Institute record example

instituteStudentPatchSchema permits name, course, branch, graduation year, CGPA, active backlogs, bio and projects. CGPA is between 0 and 10; backlogs are an integer in range; year has the expected format. XP, campus membership, account email and skill verification cannot be included. The editor sends only changed fields, reducing accidental overwrites of unchanged information.

Voice practice example

The communication schema limits transcript length and requires enough words for feedback. A typed response uses a null voice duration. Voice takes have a bounded duration. These rules prevent a typed answer from falsely obtaining a measured speaking-pace estimate.

"We validate for usability in the browser and again for safety on the server. The shared schema makes rules consistent, while authorization separately decides whether this account may make the change."

Important distinction

Validation asks, "Is CGPA 8.4 a valid value?" Authorization asks, "Can this institute change this student's CGPA?" A valid input can still be forbidden for the caller.

15 / Charts, icons, sliders, fonts and motion

Technology	Use, concepts and tradeoff
Recharts	Responsive React charts. Understand series, axes, scale, tooltip and the meaning of each measurement. A chart cannot turn sample values into real analytics.
Lucide React	Consistent SVG icons for actions and navigation. SVG scales without becoming blurry. Add accessible text or an aria-label when an icon is the only visible label.
Framer Motion	Motion components, entrance/exit behavior, scroll transforms and motion preferences. Animation should clarify a change and remain optional for reduced-motion users.
Swiper	Carousels and responsive card sliders. Keyboard, pagination and accessibility modules support navigation. It is a UI library, not a data store.
@fontsource/inter	Bundles Inter font files and weights locally. Typography stays consistent without requiring a separate font-hosting request for those assets.
Rive React Canvas	Optional interactive character driven by a Rive state machine. It is dynamically loaded, with an illustration fallback and reduced-motion handling.

CSS motion versus a motion library

The sidebar uses CSS transitions because it animates simple width and shadow changes. More complex entrance, exit, pointer or scroll relationships use React motion logic. Picking the smaller appropriate tool keeps a straightforward interaction easier to maintain.

Rive concepts

A .riv file contains authored animation data. A state machine moves among states in response to inputs, such as typing, password focus, success or error. The React canvas runtime draws it. The code has a configured enable switch and fallback behavior; installing the package alone does not mean that every login screen always runs the animation.

Performance concepts

Lazy loading avoids shipping optional heavy UI code before it is needed. `requestAnimationFrame` coordinates visual updates with the browser's rendering cycle. A responsive chart needs a container with useful dimensions. Large animation assets should not delay essential forms.

"We use Lucide for consistent icons, Recharts for data visualization, Swiper for card sliders, and Framer Motion for selected interactions. Simple sidebar motion stays in CSS."

16 / Node.js, npm and the event loop

Node.js is the runtime

Node executes JavaScript outside the browser. CampusLink uses Node 24.x for the Express server and frontend build/development tools. Built-in modules provide crypto, filesystem access, URL handling, async context and SQLite support.

Concepts you should understand

- **Runtime versus framework:** Node runs code; Express organizes an HTTP application on that runtime.
- **Event loop:** schedules callbacks and asynchronous work. Awaiting a network request does not require blocking every other request.
- **CPU-bound versus I/O-bound:** text computation uses CPU; waiting for database or provider responses is I/O. Heavy synchronous work can reduce responsiveness.
- **Built-in modules:** `node:crypto`, `node:fs`, `node:path`, `node:async_hooks` and `node:sqlite` are used in the backend.
- **AsyncLocalStorage:** carries the current authenticated workspace context through asynchronous calls, without passing it through every function argument.
- **Process environment:** server configuration is read from environment variables; `dotenv` loads a local server environment file.
- **Graceful shutdown:** stop timers, close the HTTP server and release database connections.

npm and the package files

`package.json` describes direct dependencies and named scripts. `package-lock.json` locks the dependency tree. `npm install` resolves and installs dependencies; `npm ci` uses the lockfile for repeatable clean installation and expects it to agree with `package.json`. `node_modules` is generated dependency content, not code to submit as your own implementation.

Project commands

<code>npm run dev</code>	Frontend development server
<code>npm run server:dev</code>	Backend development runner
<code>npm run server:build</code>	Bundle backend entry points
<code>npm run server:start</code>	Run the built Express API
<code>npm run typecheck</code>	Frontend/shared type checking
<code>npm run server:typecheck</code>	Backend type checking

Build output

esbuild produces `server/dist/*.cjs` bundles and source maps. CommonJS and ES modules are module systems; this server's build output uses CommonJS, while source files use modern imports. `tsx` provides TypeScript development support. Neither is the business API itself.

"Node gives our backend the runtime, network and storage APIs. npm manages the dependencies and scripts. Express sits on Node to handle HTTP requests."

17 / Express and middleware

How Express handles a request

Express matches a request to middleware and a route. Middleware can examine the request, add response headers, authenticate a user, or stop the request with an error. A route produces a response. Registration order matters because earlier middleware runs before later handlers.

Middleware present in CampusLink

Middleware or handler	Responsibility
express.json	Parses JSON bodies with a configured size limit.
cookie-parser	Exposes incoming cookie values to authentication logic.
helmet	Adds security-related HTTP headers.
cors	Permits the configured frontend origin and credentialed requests.
express-rate-limit	Separate limits for general, authentication and AI requests.
Origin checks	Reject unexpected browser write origins.
Authentication / CSRF	Require a valid server session and appropriate write token.
Zod schemas / policy	Validate payloads and restrict the service to permitted roles.
Error handling	Converts expected and unexpected failures into controlled responses.

Route parameters, body and query

`req.params` identifies a value in a route, such as a document ID. `req.query` carries URL query data, such as a search string. `req.body` holds submitted JSON or parsed form content. All are untrusted input until validated.

HTTP response codes

200 is a successful response; 201 commonly means creation. 400 is malformed or invalid input. 401 means authentication is missing or invalid. 403 means a known caller is not permitted. 404 means a resource or endpoint is unavailable. 409 describes a conflict; 422 describes a domain operation that cannot proceed. 500 represents an unexpected server failure. Use the code and a helpful message together.

Error boundaries are different

A React error boundary handles certain component-rendering errors. Express error handling handles server request failures. A backend failure does not become safe merely because the frontend catches it; the server must still reject invalid operations before data changes.

"Middleware handles common concerns such as parsing, authentication and security headers. The service layer handles the actual business rule, and controlled errors explain why an operation failed."

18 / HTTP, Axios, JSON and service adapters

The actual API pattern

CampusLink has ordinary HTTP endpoints for authentication, files and analytics, plus a service-dispatch pattern: POST /api/v1/services/:service/:method with a JSON args array. This is an RPC-style interface over HTTP; describing every method as a pure resource-style REST endpoint would be inaccurate.

Request lifecycle example

```
Institute editor -> campusService.updateStudent(id, patch)
-> service proxy -> Axios POST with JSON arguments
-> Next.js /api/v1 rewrite -> Express
-> session + CSRF + role + ownership + Zod checks
-> transaction -> updated Student JSON
-> query cache update -> refreshed interface
```

Axios responsibilities

The shared client defines the API base path, a 15-second timeout and withCredentials so eligible cookies accompany requests. Request interceptors attach headers such as Accept and the CSRF token. Response interceptors normalize failures into useful messages. Keeping this central avoids slightly different HTTP behavior in every feature.

Important concepts

- **JSON:** a text representation of objects, arrays and primitive values used for request and response bodies.
- **GET versus POST:** reads versus submitted operations; the service adapter uses POST even for some service reads.
- **Headers:** metadata such as Content-Type, Origin and X-CSRF-Token.
- **Cookie:** browser-managed session information sent according to cookie rules.
- **Timeout:** bounds how long a request may wait; it does not itself prove that a server operation was never committed.
- **Retry and idempotency:** retrying a write can repeat it unless the server deduplicates or guards the operation.
- **API contract:** expected argument shapes, response fields and failure conditions.

API and demo modes

NEXT_PUBLIC_APP_ENV selects the adapter. In demo mode, the local service writes to the mock workspace. In API mode, remoteService proxies methods to Express. This separation helps preserve the same feature interface while changing where operations execute.

"The component calls a typed service, rather than manually building requests everywhere. The adapter chooses local demo behavior or the protected API, and Axios handles the shared transport concerns."

19 / Password authentication and sessions

Authentication answers: who are you?

The backend verifies credentials and issues an opaque session token. The browser receives a `campuslink_session` cookie with HTTP-only protection. The server stores a SHA-256 digest of the session token, the user ID, CSRF token and expiry. It authenticates later requests against the database record, rather than trusting a role sent by the browser.

Password handling in this code

`server/auth.ts` uses `scryptSync` with a random salt and a 64-byte derived key. It stores salt and password hash together. Password comparison uses `timingSafeEqual` after checking length. SHA-256 is used for random token digests; it is not the password-hashing algorithm in this project.

Concepts and reasons

- **Hashing:** a one-way transformation. A password should not need to be recovered as plain text.
- **Salt:** a random per-password value that prevents equal passwords from producing identical stored hashes.
- **Password derivation function:** intentionally expensive work makes large-scale guessing harder than a fast general hash.
- **Opaque token:** a random identifier whose meaning is held by the server.
- **HTTP-only cookie:** prevents ordinary page JavaScript from directly reading the cookie.
- **Secure flag:** limits cookie transport to HTTPS when enabled for production.
- **SameSite=Lax:** reduces certain cross-site cookie sending while supporting ordinary navigation flows.
- **Expiry and revocation:** sessions have lifetimes and can be removed at logout or password reset.

Verification and reset links

Verification and password reset use random, purpose-specific, expiring tokens whose digests are stored. Consuming a token is transactional and removes it. Password reset also revokes the account's earlier sessions. These are different operations from simply hiding a login button.

JWT comparison

A JWT can contain signed claims. This project instead uses random database-backed sessions, making revocation straightforward. Do not say JWT or `bcrypt` is used here: neither is the current implementation shown in `auth.ts`.

"Passwords use salted `scrypt` hashes. Successful login creates a server-backed session in an HTTP-only cookie. Each protected request resolves the current account from that session."

20 / Authorization, RBAC and institute ownership

Authorization answers: what may you do?

Having a valid login does not permit every operation. CampusLink combines role-based access control with record ownership. Role policy checks whether a student, recruiter or campus team can call a method. Scope checks then decide which student or drive the account may access.

Three main scopes

Role	Authorized scope in the backend
Student	Own workspace and permitted student operations.
Campus / institute	Approved students and placement records belonging to its campus ID.
Recruiter	Its own drives and the applicant pool connected to them.

Administrator access is an additional server-issued permission. It is not granted by merely selecting a campus role. Approval, verification and onboarding rules also control when business operations are available.

Institute student editor walkthrough

The API accepts an explicit student ID and a narrow patch. It finds the actual student account, requires an approved student with the same non-empty campus ID as the institute, and then reads the student's persisted workspace. It applies allowed changes in a database transaction. A name change also updates the account display name. An audit entry records the institute actor, target student, changed field names and time.

Protected fields and safe updates

The editor does not assign identity, campus membership, XP or verified skills. A strict schema rejects those fields even if a caller manually modifies the request. Unrelated workspace records, contest history and points are preserved. Sending an ID alone never proves ownership.

IDOR and mass assignment

Insecure direct object reference (IDOR) occurs when a caller can access someone else's record by changing an identifier. **Mass assignment** occurs when an API blindly copies unapproved fields from a request. Campus ownership checks address the first; the patch allowlist addresses the second.

"We check both the role and the record's ownership on the server. An institute can edit academic and profile fields only for its own students, and protected fields are rejected even if someone bypasses the interface."

Defense in depth

Hiding an edit button improves the interface but is not authorization. The protected API remains the enforcement point. The scoped database metadata and server policy work together; neither should be replaced by a browser-stored role.

21 / Web security concepts in this project

Different problems need different controls

- **CSRF:** a malicious site tries to make a signed-in browser submit an unwanted action. CampusLink checks a session-specific write token and the browser origin.
- **CORS:** controls which browser origins can access certain responses. It is not a replacement for authentication or permission checks.
- **XSS:** untrusted script executes in the page. Render plain text safely and avoid unsafe HTML insertion. HTTP-only cookies help reduce token exposure but do not eliminate all XSS consequences.
- **SQL injection:** input is treated as executable SQL. Parameter placeholders bind values separately from the query structure.
- **Path traversal:** an input escapes the intended upload directory. Safe storage-key patterns and resolved-path checks protect local storage.
- **Secret exposure:** server keys end up in the browser or public repository. Server credentials belong in private server environment settings.

Controls visible in the implementation

Helmet supplies security headers. Express rate limits apply to general requests, authentication and AI operations. JSON and uploads have size bounds. The API uses no-store cache headers for private data. Uploads are checked using file signatures rather than trusting a filename. Unexpected server failures return a controlled message rather than infrastructure details.

TLS and database connections

HTTPS encrypts traffic in transit and authenticates the remote server through certificate validation. PostgreSQL supports verified TLS settings; this project can load a database CA certificate while keeping rejectUnauthorized enabled. A CA certificate can be public, while the database password and provider secret keys must stay private.

Browser storage and public variables

Values prefixed NEXT_PUBLIC_ can enter the frontend bundle. Use them for public mode or API-path configuration, not secret keys. Browser localStorage is suitable for this demo's data but is not secure server ownership enforcement. Production sessions and private files rely on the backend.

Tradeoffs to acknowledge

Security controls reduce specific risks; passing tests is not proof of complete security or production certification. The current shared-record persistence and global mutation lock deserve load and operational review before large-scale rollout. Explain the actual protections without claiming the project is impossible to attack.

"We use validation, server-side roles and ownership, CSRF and origin checks, secure cookies, private file access, parameterized SQL and bounded requests. Each control protects a different part of the request path."

22 / SQLite, PostgreSQL and the repository layer

Two database implementations

SQLite is an embedded relational database useful for local development and isolated tests. Node's DatabaseSync opens a local file or an in-memory database. PostgreSQL is an external database server useful for a hosted deployment. The pg library provides SQL connections and a pool. CampusLink switches between them through server/db.ts.

What is actually stored

The migration creates a generic records table with kind, id, campus_id, owner_id and value. The value contains serialized JSON text. A composite primary key identifies (kind, id), and an index supports scoped lookups. There are separate lock and migration-tracking tables. This is not a fully normalized collection of separate student, contest and offer SQL tables.

Database concepts

- **Relational database:** organizes stored data through tables and defined keys.
- **Primary key:** identifies a row uniquely.
- **Composite key:** uses more than one field to identify a row.
- **Index:** accelerates suitable reads but adds storage and write overhead.
- **SQL:** the query language used to select and modify rows.
- **Parameterized query:** placeholders carry values without making them part of executable query syntax.
- **Upsert:** inserts a new row or updates an existing key; db.put uses ON CONFLICT.
- **Connection pool:** reuses a bounded set of PostgreSQL connections instead of reconnecting for every operation.
- **Migration:** a repeatable schema setup step tracked in the database.

The repository API

db.get reads one record; db.list reads a scoped collection; db.put saves; db.remove deletes; db.transaction coordinates related changes. The same business logic can call these methods without deciding the SQL driver each time.

Tradeoff of JSON-valued records

This approach is flexible for evolving workspace objects and a shared local/hosted adapter. It gives fewer database-level constraints for nested fields and can make fine-grained reporting or partial updates more expensive. Zod and domain rules enforce many nested-data constraints today. A later normalized design could add stronger foreign keys and more targeted queries.

"The current database schema stores typed application records as JSON text with campus and owner metadata. SQLite supports local use; PostgreSQL uses the same repository interface for hosted persistence."

23 / Transactions, concurrency and audit trails

Why a transaction is needed

A business operation often changes more than one record. An institute name change updates both the workspace profile and account display name. An award changes contest completion, history and XP. A transaction keeps related database work together so the operation commits as a unit or rolls back after a failure.

ACID in simple language

- **Atomicity:** all related writes succeed or the operation is rolled back.
- **Consistency:** valid business and database rules remain satisfied before and after the change.
- **Isolation:** simultaneous operations do not unexpectedly interfere through partial changes.
- **Durability:** committed changes remain in persistent storage according to the database's guarantees.

The concurrency problem

Two requests might both read "contest incomplete," then both add points. Two scheduling operations might both find the same hall free. Checking only in the browser cannot prevent this race. The critical read-and-update sequence must be guarded on the server and inside the correct transaction or lock.

How the current adapter handles it

For PostgreSQL, the adapter begins a transaction and locks the placement lock row using `SELECT ... FOR UPDATE`. For SQLite, a queued mutation runs with `BEGIN IMMEDIATE`. `AsyncLocalStorage` allows nested operations to share their active transaction. This conservative global serialization prevents several races but reduces parallel write throughput across unrelated campuses.

Idempotency and replay protection

An idempotent operation has the same business effect when repeated. Contest completion checks the persisted completed flag and tagged history before adding points. Submitted assessment and interview sessions also guard repeated completion. This is essential when a browser retries after a slow or interrupted response.

Audit trail

Audit records describe who performed an operation, which student was affected, the event and time. The institute update records field names, not a complete duplicate of sensitive profile values. Auditing supports review and diagnosis but does not replace authorization.

"We guard the read-and-write sequence transactionally. Repeated contest submissions cannot award points twice, and institute edits retain an audit record of the actor and changed fields."

Scaling follow-up

Per-campus or per-resource locks would allow more parallel work. That is a future design improvement, not the locking strategy implemented today.

24 / Supabase and Google sign-in

What Supabase provides in this design

Supabase can host PostgreSQL, private storage, and the Google authentication provider flow. CampusLink talks to these capabilities through server-side SQL or HTTP integrations. The application does not use a Supabase browser SDK as the central data layer, and it does not let the browser directly read private workspace rows.

OAuth and PKCE concepts

OAuth coordinates delegated access; an identity provider supplies the verified identity used by a sign-in flow. PKCE protects the authorization-code exchange. The application generates a secret verifier and sends its hashed challenge when starting the flow. Later it presents the verifier while exchanging the returned code. A separate state value binds the callback to the browser that started it.

The implemented Google flow

server/google.ts checks provider setup. The server stores a short-lived flow record with the verifier and chosen role, and sets an HTTP-only flow cookie. It redirects through Supabase's Google provider. The callback checks the cookie/state relationship, consumes the flow record once, performs the code exchange, and verifies the resulting identity. It then issues CampusLink's existing server session.

Identity is not institute authority

Google proves an identity; it does not automatically approve a recruiter, grant administrator access, or establish a student's institute membership. Existing users retain their role and approval state. New accounts complete onboarding. Campus and recruiter approval rules still apply. This separation prevents social login from becoming a privilege shortcut.

Row-level security and the server boundary

PostgreSQL application tables have RLS enabled with no browser-facing policies. The backend uses its configured database connection and applies application scope checks. A privileged server connection must still enforce those checks; enabling RLS by itself is not proof that every connection is automatically tenant-safe.

Configuration versus live verification

Google credentials and redirect settings must be configured in the provider dashboards. Storage needs a private bucket and server secret. PostgreSQL needs its connection and TLS settings. This guide describes the implemented integration, not a claim that the hosted services were all tested with live credentials during PDF creation.

"Google sign-in is mediated through Supabase using a server-side PKCE flow. After identity verification, our own session and role/approval rules still control the CampusLink workspace."

25 / File uploads, Multer and PDF parsing

From browser file to private record

The browser submits multipart form data. Multer parses the file for Express. The server enforces an upload size bound and validates allowed signatures. It generates an owner-specific storage key, stores the bytes locally or in a private Supabase bucket, and associates document metadata with the student's workspace. Downloads go through an authorized API route.

Concepts and responsibilities

- **Multipart form data:** an HTTP format that can carry file bytes and form fields together.
- **MIME type:** a label such as application/pdf; a client-supplied label alone is not reliable validation.
- **Magic bytes / signature:** initial byte patterns identify supported file formats.
- **Metadata versus bytes:** a database record can describe a file while the content lives in separate storage.
- **Generated storage key:** a server-generated identifier avoids treating an original filename as a trusted path.
- **Private bucket:** storage content is not exposed as an unrestricted public asset.
- **Authorized download:** the API checks the account before returning content.

Resume analysis

pdf-parse extracts text from text-based PDF resumes. The local NLP pipeline then finds skills and suggests clearer evidence. A scanned image-only PDF usually needs optical character recognition, or OCR. An OCR integration is not connected in the current repository. PDF text extraction is not the same operation as interpreting a photograph.

Failure and cleanup

Uploads can fail after bytes are written but before all records commit, so transactional design and cleanup jobs matter. Removed stored file objects can be queued for cleanup after record changes, avoiding deletion of a file before a related transaction succeeds.

Local and hosted storage

Local files live outside public static assets and are checked using safe path resolution. Hosted mode uses a configured Supabase private bucket with server-side credentials. Switching the storage provider does not automatically migrate older local uploads; record and content migration need deliberate handling.

"Multer handles the upload transport. Our server validates the content and ownership, stores it privately, and keeps metadata in the workspace. pdf-parse extracts resume text for analysis; scanned resumes need an additional OCR solution."

26 / Email, the outbox and reminders

Why email uses a job record

A placement event can create an in-app notification and an email job. Saving a job lets the application retry delivery if the provider is unavailable. It also prevents a slow email request from being the only record that a notification should exist.

The implemented components

server/mail.ts queues jobs with recipient, subject, text, status, attempts and the next scheduled attempt time. When EMAIL_PROVIDER is resend, a worker calls the Resend HTTP API using a server key. In local outbox mode, pending mail can be inspected without sending messages externally. Notification records are scoped to recipients.

Concepts used

- **Outbox pattern:** persist a pending delivery alongside the business event.
- **Worker:** processes pending jobs after the operation that created them.
- **Deduplication:** an event key avoids creating the same logical job repeatedly.
- **Idempotency key:** the provider receives the job ID to reduce repeated sends during retry.
- **Exponential backoff:** retry delays grow after repeated failures instead of hammering the service.
- **Retry limit:** after a bounded number of failures, the job becomes failed for review.
- **Reminder scheduling:** identifies upcoming interviews or document/joining work and creates appropriate notices.

Current execution model

server/index.ts runs mail delivery, reminders and storage cleanup on an interval in the API process. This is a database-backed worker loop, not a BullMQ/Redis queue or an independently hosted scheduler. Multiple instances and crashes need careful delivery semantics. A production scaling improvement would move workers out of the web process.

Hosted configuration

Resend needs a private API key and verified sender configuration. An always-running host is useful when scheduled delivery must be reliable. A sleeping or stopped API process cannot keep running its own interval workers.

"We queue recipient-specific email jobs and retry them with deduplication and backoff. Local development uses an outbox; configured hosted delivery uses Resend. Reminders currently run inside the API process."

Claim accurately

An email job being queued does not prove delivery. The job status and provider response determine whether it was sent. Test outbox behavior and actual provider delivery separately.

27 / Recruitment workflow and state machines

A workflow is more than a status label

A state machine describes valid states and permitted transitions. A recruitment drive cannot jump from a private draft to a completed public drive just because someone changes a string. CampusLink validates the role, current state, required schedule and confirmation before allowing the next step.

Core drive sequence

```
DRAFT -> SUBMITTED -> UNDER_REVIEW -> SCHEDULING
-> AWAITING_RECRUITER_CONFIRMATION -> CONFIRMED
-> campus finalization -> ACTIVE -> IN_PROGRESS -> COMPLETED
```

Review changes, rejection, cancellation and resubmission have additional guarded paths. Recruiters request visits; campus teams review and propose schedules; recruiters confirm; campuses finalize and activate. Students see eligible opportunities derived from activated drives.

Eligibility gates

Campus, course, branch, graduation year, minimum CGPA and active backlog constraints are checked before applying or matching. Eligibility is a hard gate. A high text similarity or high XP cannot override a failed academic or institute requirement.

Scheduling conflicts

The schedule domain checks overlaps in relevant resources, recruiter timing and student cohorts. A transaction guards conflicting simultaneous reservations. A schedule editor's visual calendar is not enough: the API must validate the request against the latest records.

Application and offer lifecycle

Applications move through defined recruitment stages. Recruiters can create an offer after the relevant selection stage. Students can respond using allowed transitions such as acceptance or decline. Campus teams verify documents and record joining. Joining requirements include the relevant verified documents; an accepted offer and a joined record are different states.

Why state machines are useful

They prevent impossible sequences, make the UI show the next valid action, and give tests a clear set of allowed and forbidden transitions. They also preserve accountability about which role performs each step.

"The system models recruitment as guarded state transitions. Matching and applications use eligibility checks, and a drive becomes a student opportunity only after the campus scheduling and confirmation process activates it."

Follow-up to practice

Where should transition rules live? In shared domain logic for consistency and in the protected server operation for enforcement. A frontend button can guide the user, but it cannot be the only guard.

28 / Assessments, grading and verified skills

Purpose

Assessments turn a student's claimed skill into recorded evidence. The API starts a session, sends questions without exposing answer keys, accepts responses, computes the score, records the attempt and awards the configured points. The service checks completion state so a submitted session cannot be replayed for repeated rewards.

Concepts used

- **Question bank:** the source of questions and correct answers.
- **Assessment session:** a record that connects a start time, selected questions and submission to the student.
- **Server-side grading:** the backend compares answers against the authoritative question data.
- **Score:** performance on that attempt, often expressed on a 0-100 scale.
- **Verification threshold:** a configured condition for marking a skill verified after a successful skill assessment.
- **Attempt history:** preserves name, score, points, date, duration and related identifiers.
- **Replay protection:** prevents a completed session being submitted again to award points.

Why grading should be on the server

The browser is inspectable. If the correct answer is shipped as part of the active API question response or if the caller supplies their own score, a user can fabricate results. CampusLink's API tests check that the response does not reveal the answer key and that replay is rejected.

Points and verified skills are different

XP represents activity rewards. Verified status represents a result from the skill verification path. Institute profile editing cannot arbitrarily set either of these protected values. Points do not guarantee a placement outcome, and completing a general contest is not necessarily the same as a particular skill verification assessment.

Demo and API distinction

The local adapter supports demonstration flows using sample data. API mode stores sessions and results on the server and applies authorization and submission guards. In a viva, state clearly which mode you are demonstrating.

"The API grades the submitted answers and records the result. It hides answer keys from active question responses and prevents the same completed session from generating repeated rewards."

Further improvements

Question randomization, broader banks, moderation, accessibility accommodations and formal assessment validity can be improved later. A sample quiz result alone is not evidence of a comprehensive certified evaluation.

29 / Readiness, eligibility and hybrid matching

Three different ideas

Eligibility decides whether the drive's hard requirements are met. Readiness summarizes preparation evidence. Matching ranks eligible students or opportunities using relevant evidence. They are not interchangeable scores.

Readiness weights in src/utils/scoring.ts

Category	Weight and current evidence
Verified skills	30%; proportion of listed skills marked verified.
Academics	20%; CGPA scaled to 100 with a backlog adjustment.
Projects	15%; capped project-count contribution.
Aptitude	15%; relevant recorded attempt scores.
Communication	10%; communication/soft-skill assessment history.
Interview	10%; interview attempt history.

Missing evidence is tracked separately from an actual zero-score attempt. Voice practice feedback is saved in its own practice history; it does not automatically become a communication assessment score in this readiness calculation.

A worked weighted-score example

For illustrative category values 80, 85, 70, 60, 50 and 75, readiness is $80 \cdot 0.30 + 85 \cdot 0.20 + 70 \cdot 0.15 + 60 \cdot 0.15 + 50 \cdot 0.10 + 75 \cdot 0.10 = 73$. The calculation combines recorded category values; it is not a machine-learned placement probability.

Matching weights

Rule fit combines skill alignment (45%), assessment performance (20%), project relevance (15%), certifications (5%) and academics (15%). Candidate ranking can then combine 85% rule fit with 15% TF-IDF text relevance. Example: rule fit 80 and relevance 60 gives $\text{round}(80 \cdot 0.85 + 60 \cdot 0.15) = 77$. If eligibility fails, the matching score is gated to zero.

Limits worth explaining

Count-based project evidence does not automatically prove quality. A text match does not deeply understand competence. Weights are product rules until validated against an appropriate evaluation. The UI should explain evidence and gaps rather than presenting a score as a guarantee of employment.

"Readiness is a transparent weighted preparation score. Our hybrid ranking combines rule evidence and lexical relevance only after eligibility passes. It is separate from the optional trained outcome model."

30 / NLP, TF-IDF and cosine similarity

NLP in this repository

Natural language processing means extracting useful structure from text. `server/nlp.ts` uses an alias dictionary, boundary-aware regular expressions, normalization, stop-word removal and vector similarity. It recognizes variations such as `reactjs` and `React` as the same skill label. It also extracts explicit CGPA, branch and graduation-year criteria for human review.

The processing steps

- **Normalization:** lowercase text and map recognized aliases to consistent forms.
- **Tokenization:** split text into meaningful word-like units.
- **Stop words:** remove common words that add little matching information.
- **Vocabulary:** the set of terms across the query and candidate documents.
- **Term frequency (TF):** how often a term occurs relative to the document's token count.
- **Inverse document frequency (IDF):** gives rarer terms more influence across the compared corpus.
- **Vector:** a numeric representation of term weights.
- **Cosine similarity:** compares vector directions, giving a relevance measure independent of raw document length.

Formula used by the local implementation

```
TF(term, document) = occurrences / token_count
IDF(term) = ln((N + 1) / (document_frequency + 1)) + 1
TF-IDF = TF * IDF
cosine(A, B) = dot(A, B) / (length(A) * length(B))
```

If either vector has zero magnitude, the local similarity function returns zero rather than dividing by zero. Skill aliases reduce some vocabulary mismatch, but this remains lexical matching, not a general semantic embedding model.

Resume and interview analysis

Resume analysis looks for recognized skills, measurable outcomes and project/professional-link context. Interview feedback uses response detail, topic relevance, structural keywords and evidence. These are transparent heuristics. Text with different wording can have the same meaning, while keyword repetition can inflate lexical relevance.

Why this choice is useful

It runs without an external model key, is inspectable, and gives a reliable fallback path. It is limited by the dictionary, patterns and compared corpus. Extracted requirements remain reviewable rather than automatically altering official eligibility rules.

"Our local NLP uses keyword normalization and TF-IDF cosine similarity. It is explainable and runs locally, but it is not a transformer model or a deep semantic understanding system."

31 / Logistic regression and ML evaluation

What machine learning adds

The local outcome pipeline in `server/ml.ts` learns coefficients from labeled historical rows. Six preparation features are scaled to 0-1, an intercept is added, and logistic regression produces a probability-shaped estimate. This is different from manually chosen readiness weights.

Model and training concepts

```
x = [1, verifiedSkills/100, academics/100, projects/100,
      aptitude/100, communication/100, interview/100]
z = sum(weight[i] * x[i])
p = sigmoid(z) = 1 / (1 + exp(-z))
```

The local trainer uses gradient updates over 1,800 epochs with a learning rate of 0.15 and an L2 regularization term for non-intercept weights. An epoch is one pass through the training rows. Regularization discourages overly large coefficients and can reduce overfitting.

Data requirements and split

The pipeline requires at least 60 labeled rows across three cohorts and both outcomes in training and held-out sets. The latest sorted cohort is held out. Use chronological cohort labels and features measured before the placement outcome. A person should not appear in both training and evaluation data. Otherwise leakage can make metrics look better than real performance.

Metrics returned

- **Accuracy:** fraction of correct classifications at the chosen threshold.
- **Precision:** among predicted positive outcomes, how many are actually positive.
- **Recall:** among real positive outcomes, how many are identified.
- **Brier score:** average squared error between probabilities and 0/1 outcomes; lower is better.
- **Calibration:** whether outcomes occur roughly as often as stated probabilities; Brier score is informative but is not a complete calibration assessment.

Provenance and limits

The saved model records version, weights, cohorts, metrics and whether data is synthetic or historical. Synthetic demonstration models are blocked for production inference. Historical coefficients show associations, not causation or a guaranteed personal outcome. Local ML is implemented in TypeScript, not through scikit-learn, TensorFlow or PyTorch in this repository.

"We have a trainable logistic-regression pipeline with a chronological cohort holdout. I would only discuss real-world predictive quality after evaluating a consented historical dataset; synthetic demo metrics demonstrate the pipeline, not deployment accuracy."

32 / Optional OpenAI coaching and external ML

Three separate intelligence paths

Local rules and NLP work without an AI provider. Optional OpenAI coaching adds generated preparation advice. An external ML API adapter can supply compatible model or text-analysis responses. These paths have different data, configuration and consent requirements.

The OpenAI integration actually used

server/generative.ts calls the Responses API using native server fetch, not an installed OpenAI JavaScript SDK. The configured default model name is gpt-4.1-mini, which can be changed through OPENAI_MODEL. The server requests a strict JSON schema with summary and suggestions, bounds the output budget, uses a timeout and validates the parsed result with Zod.

Concepts to understand

- **Generative model:** produces advice based on a task and supplied evidence.
- **Prompt / instructions:** describe the intended coaching task and output behavior.
- **Token budget:** limits generated output; it affects latency, cost and response length.
- **Structured output:** requests a defined response shape; shape compliance does not prove factual correctness.
- **Grounding:** provide actual project or student evidence instead of inventing achievements.
- **Prompt injection:** untrusted evidence tries to act as an instruction; the coaching instructions treat evidence as data.
- **Fallback:** preserve available local analysis if the external provider fails.

Consent and privacy

The student must opt in before the OpenAI path sends practice or resume evidence externally. The backend redacts common email and phone patterns, omits unnecessary identity fields, and sets store:false in the API request. This flag is not a promise about every aspect of provider retention or the browser's speech service. Advice does not directly change official placement records.

External ML adapter

server/ml-client.ts uses a configured API URL and bearer token, an approved destination, bounded timeout and response-size limits. It validates returned schemas and falls back on unavailable, invalid or unverified responses. Remote placement responses must identify historical provenance; a compatible JSON shape alone is insufficient.

"OpenAI is an optional coaching layer behind student consent. It generates advice, while the application retains deterministic validation, permissions and eligibility decisions. Provider failure leaves the local feedback usable."

Official references: [OpenAI text generation](#) and [Structured outputs](#).

33 / Voice communication practice

What the feature does

A student selects a prompt, speaks through browser recognition or types an answer, reviews the transcript, and asks for feedback. The feature highlights possible wording errors and corrections, filler words, repeated phrases, answer structure and improvement tips. The latest 15 takes can be reopened.

The actual speech technology

The browser's SpeechRecognition or webkitSpeechRecognition implementation converts speech to text. The code requests English with en-IN, handles interim and final results, permission/error states, stop/reset and a 180-second take limit. CampusLink does not implement its own audio model or use OpenAI Realtime/Whisper as the current microphone path.

Key concepts

- **Speech-to-text:** converts an audio input into words.
- **Interim transcript:** a provisional recognition result that can change.
- **Final transcript:** a result marked final by the recognizer.
- **Recognition lifecycle:** start, result, error, end, stop and cleanup.
- **Fallback input:** students can type when browser recognition is unavailable.
- **Words per minute:** approximate pace from word count and valid voice duration, not a measure of speaking quality.

Feedback pipeline

The transcript is validated before analysis. Local checks identify a limited set of grammar/wording patterns, fillers and response structure. API mode stores the result under the submitting student's practice history. Optional consented OpenAI coaching can add advice; failure keeps local checks. Text edits clear the voice duration so modified text is not presented as a measured voice sample.

Boundaries to explain correctly

The browser may use a remote speech-recognition service and therefore send audio there. The feature asks for speech consent before starting. CampusLink stores transcript and feedback rather than uploading or keeping an audio recording. The transcript can contain recognition mistakes. The feature does not evaluate pronunciation, accent, confidence, emotion or acoustic pauses.

Validation status

Transcript saving, local error feedback and responsive layout were exercised in the browser. Live microphone hardware was not verified during the earlier implementation checks. Browser speech support and permissions can vary. Do not present the feature as a validated spoken-language examination.

"We use browser speech recognition to obtain a transcript, then provide wording and structure feedback. Students can correct recognition mistakes before analysis. It is a communication practice tool, not a pronunciation or confidence detector."

Reference: [MDN SpeechRecognition](#).

34 / Contest points, streaks and badges

The source of truth

Achievements come from successful completion history, not from a manually editable badge field. `recordContestCompletion` marks a contest complete, adds its configured points and records a tagged contest attempt. The operation checks completion/history first so repeats cannot add the same award again.

Streak calculation

The utility groups successful completions by distinct contest and calendar date. Multiple contests on the same day count as one streak day. Consecutive dates form a run. The current streak remains extendable when the last completion was yesterday; after a missed day it resets. The longest historical streak remains available.

Calendar time matters

Dates use Asia/Kolkata, or IST, consistently on client and server. A completion just after midnight IST belongs to the new day even if UTC still shows the previous date. Date-only calculations use a consistent day representation instead of the device's arbitrary local timezone.

Badge	Requirement
First finish	Complete 1 contest.
Challenger	Complete 3 different contests.
Contest explorer	Complete 5 different contests.
Three-day spark	Complete contests on 3 consecutive days.
Weekly momentum	Complete contests on 7 consecutive days.

Derived badges and retained history

The utility derives earned dates and progress from history. A reset current streak does not erase a previously earned streak badge. New tagged completion records remain recognizable after a contest is archived. Older history can be recognized when its ID matches a stored contest. This compatibility is useful without fabricating a completion date for an undated flag.

Where people see it

Students see achievements in Contests and Career Points, with a compact dashboard link. Institutes can inspect a student's achievements through a campus-scoped service. Profile editing cannot assign XP or badges directly. Registration alone and incorrect solutions do not unlock rewards.

"Badges and streaks are calculated from successful contest records. We deduplicate completions, use IST calendar days, preserve earned milestones, and award points only once."

Useful follow-up

How can a student reach seven days? They need available distinct contests completed across seven consecutive dates. The sample contest catalog is small; more published contests are needed for longer ongoing streaks.

35 / Tests, linting, formatting and builds

What each tool checks

Tool	Purpose and useful distinction
Vitest	Executes automated tests, assertions and mocks. A passing test proves the tested case, not every possible behavior.
Supertest	Calls Express endpoints in tests with real request/response behavior, including cookies and CSRF headers.
TypeScript	Finds type incompatibilities without needing to run the whole app. It does not validate hostile incoming JSON by itself.
ESLint	Flags selected source patterns and potential mistakes. It is not a full security scanner.
Prettier	Formats code consistently. Formatting changes do not establish business correctness.
esbuild	Produces runnable backend bundles. A successful bundle is not the same as a passing integration test.
Next.js build	Compiles and packages the frontend for production execution. Development success alone does not guarantee a production build.

Test categories in this project

Pure tests exercise readiness, matching, workflow rules, voice helpers and achievement calculations. Backend tests use an isolated SQLite database and an Express app. They check role/campus scope, protected fields, files, sessions, grading, replay protection and conflicting schedules. Provider calls can be mocked to test timeouts, consent and fallback behavior.

Representative achievement and institute cases

The test suite covers repeated contest awards, several completions on one date, a missed-day reset, preserved streak badges, IST midnight, cross-institute rejection and the preservation of student points while academic data changes. These cases are meaningful because mistakes would affect privacy or earned progress.

Latest observed checks

The latest full test run in this conversation passed **156 tests in 19 files**. Frontend/shared and server type checks, source linting and the server build also passed. Browser preview checks covered saved institute edits, contest progress and phone layouts. The PDF creation task did not rerun these tests; this is the recorded implementation-check result.

Test technique vocabulary

An assertion checks an expected result. A fixture creates test data. A mock replaces an external dependency. A regression test prevents a known behavior from breaking again. A unit test focuses on a small rule; an integration test checks cooperating layers. Browser checks exercise the actual interface.

"I test the rules that matter most: ownership, valid transitions, data preservation and awarding exactly once. Type checks and linting complement those tests; they do not replace them."

36 / Deployment, environment variables and Git

Deployment topology in the repository

Vercel is configured for the Next.js frontend. Render is configured for the Express API. PostgreSQL and private file storage can use Supabase; email can use Resend. `render.yaml` and `vercel.json` are configuration, not proof that a live deployment is running.

Configuration concepts

- **Build time versus runtime:** some frontend configuration is embedded when building; server variables are read by the running process.
- **Public versus private environment variables:** `NEXT_PUBLIC_` values can be visible in the browser. Database passwords and provider keys must stay server-only.
- **Same-origin proxy:** the browser uses `/api/v1` while Next.js routes to the configured `API_INTERNAL_URL`.
- **Health check:** `/api/v1/health` tests API/database responsiveness.
- **TLS:** HTTPS and verified database certificates protect traffic in transit.
- **Persistent storage:** generated files on a temporary host filesystem may not survive restarts; the production configuration requires external private storage.
- **Production guard:** `server/config.ts` rejects a production setup relying on SQLite, local uploads or the email outbox.

Git and reproducibility

Git records source history. A branch separates a line of development; a commit records a snapshot; a pull request supports review before merging. The lockfile helps repeat dependency installation. Do not commit `.env` secrets, `server/data`, `uploads`, `node_modules` or `.next` build output. A public CA certificate is different from a private API key.

Build and release sequence

Install from the lockfile, run checks, build frontend and backend, configure hosted services, deploy, then verify login, protected requests, private uploads, workflows and provider delivery in that environment. Local tests cannot certify external service configuration or real-world load.

Operational tradeoffs

The current API hosts interval workers itself. When the process sleeps or stops, its workers pause. Multiple campuses and larger loads need monitoring, backups, restore testing, more selective locking and worker separation. These are operational concerns beyond a polished dashboard.

"The repository separates frontend and API deployment. It has explicit provider configuration and production guards. I would verify the hosted integrations after deployment instead of treating configuration files as evidence of a working live service."

37 / Feature walkthroughs from screen to storage

A. An institute changes a student's CGPA

1. PeoplePage fetches the campus-scoped student list using a role/account-specific query key. The institute opens InstituteStudentEditor for a chosen student.
2. The editor validates its values, selects changed fields, and calls `campusService.updateStudent(studentId, patch)`. API mode transports the request through Axios and the service proxy.
3. Express checks the session, CSRF, campus role and strict patch schema. The service resolves the actual student account and checks its campus ID against the institute's campus ID.
4. The transaction reads that student's workspace, applies allowed fields, preserves unrelated history and progress, updates the account display name if needed, and writes an audit event. It returns the saved student. The client updates and invalidates caches.

B. A student completes a contest

1. The student registers and submits the solution. The backend obtains the contest visible to that student and checks the correct answer; managed contests use their stored answer.
2. The protected workspace mutation checks whether completion was already awarded. On first completion it adds the configured points, marks completion and writes an explicitly tagged history record using the IST date.
3. The shared achievement utility deduplicates contest history, groups calendar days, computes current/longest streaks and derives badge milestones. ContestProgress displays earned dates and next-badge progress. Repeating the request does not add more points.

C. A student practices communication

1. CommunicationPractice selects a prompt. `useVoicePractice` runs browser recognition after speech consent, or the student types an answer.
2. The student reviews the transcript. The schema validates it; local analysis produces corrections, metrics and tips. API mode may add consented OpenAI coaching, retaining local output if it fails.
3. The server saves the feedback under that student's practice history. React Query refreshes recent takes. Another student cannot retrieve it, and institute profile editing cannot rewrite that student's practice history through its patch endpoint.

How to present a walkthrough

Name the starting screen, the called service, the enforcement point, the storage change, and the visible result. This is stronger evidence of understanding than reciting a dependency list.

38 / Troubleshooting and design tradeoffs

Symptom or question	Explain or investigate
Saved data is not visible	Check whether the save succeeded, the right student was targeted, and the relevant query was updated or invalidated.
A campus edit is forbidden	Confirm the server identity, approval and ownership; a valid student ID alone does not grant access.
Microphone feature is unavailable	Check browser recognition support, consent, permission and service availability; use typed input as fallback.
Pace estimate disappears after edits	This is intentional: an edited transcript no longer represents the timed voice take.
A badge does not unlock on registration	Successful completion is required; registration and wrong answers do not count.
Streak resets unexpectedly	Check calendar dates in IST and whether there was a missed day; multiple contests in one day count once.
AI advice is missing	Check provider mode, student consent, credentials, timeout and response validation. Local analysis should remain usable.
A hosted database connection fails	Check connection address, credentials, network access and verified TLS/CA configuration without exposing secrets.
A resume has no extracted text	It may be scanned; the current parser does not provide OCR.
Email remains pending	Check worker execution and provider setup; a queued job does not imply successful delivery.

Choices and future improvements

The generic record store is flexible, but a normalized schema can strengthen data constraints and reporting. The global transaction lock is simple, but per-resource locks improve concurrency. Client-driven workspace routing is practical here, but selected server-rendered views could reduce client work later. Local NLP is inexpensive and explainable, but richer semantic models need careful evaluation and privacy controls.

Performance concepts

Cache appropriate reads, lazy-load optional visuals, avoid duplicate requests, index frequent database scopes, and measure rather than guess. Pagination can reduce large student-list payloads. CPU-heavy analysis and provider calls should not hold a broad placement lock unnecessarily. Backups and monitoring are separate from frontend speed.

"I can explain why a design was chosen and its limits. The next improvements would come from measured needs: larger student pools, more concurrent mutations, and stronger evaluation of the preparation feedback."

39 / Interview questions: frontend

1. Why React and Next.js together?

React provides components and stateful UI. Next.js supplies the application framework, routes, build process and proxy configuration. CampusLink uses a client-driven Platform component within the App Router and a separate Express business API.

2. Why use TypeScript?

It catches incompatible values during development and documents contracts such as Student and Contest. Since types disappear at runtime, the server still validates incoming JSON with Zod.

3. Why both Zustand and TanStack Query?

Zustand holds small shared session state. Query manages asynchronously fetched records, caching and freshness. React state holds local UI values such as modal visibility and input text.

4. How does a saved profile update the screen?

The API returns the saved student. PeoplePage updates the affected cached list entry, invalidates the list/ranking queries and refreshes the workspace. A successful write and a fresh display are related but separate steps.

5. What are props and state?

Props are inputs passed from a parent. State is owned by the component and can change through its setter. A component can receive a student prop while holding its own draft form values in state.

6. Why clean up an effect?

Listeners, timers or a recognizer can continue after a component disappears. Cleanup releases those resources, prevents duplicate handlers and avoids updates using old state.

7. How is the expanding sidebar implemented?

CSS changes its desktop width and shadow on hover or visible keyboard focus with a smooth transition. Mobile uses a drawer. The expansion overlays the page while the collapsed space remains reserved.

8. Does using Tailwind mean there is no custom CSS?

No. Tailwind is imported and built through PostCSS, but CampusLink has extensive custom theme and feature CSS. The correct explanation includes both.

9. What is hydration?

It connects interactive React behavior to markup initially delivered to the browser. Browser-only operations such as speech recognition must run on the client, with appropriate lifecycle handling.

10. Why do list keys matter?

Stable IDs tell React which rendered item corresponds to which underlying record. An index can be misleading when items are added, reordered or removed.

40 / Interview questions: backend and data

1. Why Express rather than putting everything in the frontend?

The server needs trusted identity, private credentials, authoritative validation and persistence. Express provides routing and middleware, while domain services implement the actual operations.

2. Authentication versus authorization?

Authentication establishes who the caller is. Authorization checks which action and record the caller may access. A logged-in institute is still forbidden from editing another institute's student.

3. How are passwords stored?

As salted script-derived hashes, not plain text. Random session/reset tokens use SHA-256 digests in storage. This project uses opaque cookie sessions, not JWT or bcrypt.

4. Why is CORS not enough?

CORS is a browser-origin response-access policy. It does not decide whether an account may read or change a student record. The API also checks authentication, CSRF, origin, role and ownership.

5. What is an IDOR bug?

Accessing someone else's record by changing a request ID when the server fails to check ownership. CampusLink resolves the actual account and compares institute membership before a campus edit.

6. How do you stop arbitrary XP updates?

The institute patch schema strictly allows only academic/profile fields. XP and verified skills are rejected even if a caller sends them manually. Award paths separately control progression.

7. Why transactions?

Related writes should commit together and the critical read/update must be guarded against concurrent changes. Otherwise a failed edit can leave inconsistent records or simultaneous submissions can duplicate awards.

8. Is the database fully normalized?

No. The present repository uses a generic scoped records table with serialized JSON text. This is flexible but has fewer database-level constraints for nested fields than a normalized design.

9. Why SQLite and PostgreSQL?

SQLite makes local setup and isolated tests simple. PostgreSQL supports hosted database service usage. The adapter exposes the same get/list/put/transaction interface to business operations.

10. How are private files protected?

File uploads are validated, stored outside public assets or in a private bucket, and downloaded through an authorized API route. Knowing a filename alone is not permission to read it.

41 / Interview questions: AI, voice and achievements

1. Is all the feedback generated by an AI model?

No. The app has deterministic readiness rules, local NLP and transcript heuristics. Optional consented OpenAI coaching adds generated advice; the external ML adapter is another configurable path.

2. What is TF-IDF?

It weights words by frequency within a document and rarity across the compared corpus. Cosine similarity then compares weighted text vectors. This provides lexical relevance, not deep semantic understanding.

3. What ML model is implemented locally?

A regularized logistic-regression model with six scaled preparation features and an intercept. It learns from labeled rows and evaluates on the latest held-out cohort.

4. Can you claim a placement-prediction accuracy percentage?

Only with an identified, appropriate evaluation dataset and the actual metrics from its held-out evaluation. The synthetic demonstration does not establish accuracy for real students.

5. What does voice practice measure?

Browser recognition supplies a transcript. The app checks wording, fillers, repetition and structure, with approximate pace for valid timed voice takes. It does not assess accent, pronunciation, emotion or confidence.

6. Is this an OpenAI audio feature?

The current microphone path uses the browser Web Speech API. OpenAI is an optional text-coaching integration after transcript review and student consent. CampusLink does not upload or store an audio recording for this feature.

7. Why can students edit their transcript first?

Recognition may mishear or omit words. Reviewing the transcript makes feedback more useful. Editing clears the voice-duration association because the revised text no longer reliably represents the timed audio.

8. How are badges retained after a missed day?

The current streak resets, but earned milestones are derived from the historical completion records and longest runs. Previously earned badges do not depend on the current streak staying active.

9. Why IST rather than the device timezone?

A shared calendar timezone makes the same completion belong to the same streak day on both client and server. Otherwise different devices could disagree near midnight.

10. What happens when the AI provider fails?

The app retains local feedback, reports unavailable coaching appropriately and avoids fabricating a provider result. Provider responses also need schema validation before they are used.

42 / Glossary: application and web concepts

Term	Meaning in plain language
API	A defined way for one program to request another program's data or behavior.
Adapter	A layer that exposes one interface over different underlying implementations.
Backend	The server-side application handling trusted operations.
Cache	A temporary saved result used to avoid unnecessary repeated work.
Client	A program requesting data or behavior; here usually the browser.
Component	A reusable part of the React interface.
Contract	Agreed input, output and error shapes between layers.
CRUD	Create, read, update and delete operations.
DOM	The browser's structured representation of page elements.
Endpoint	A reachable HTTP route for a defined operation.
Event	A signal such as a click, keyboard action or recognition result.
Framework	A system that organizes how an application is structured and run.
Hook	A React function connecting a component to state or lifecycle behavior.
Hydration	Connecting browser interactivity to initially delivered React markup.
Interceptor	Shared logic that runs around an HTTP request or response.
JSX / TSX	Component markup syntax embedded in JavaScript / TypeScript.
Middleware	Request-processing logic that runs before or between route handlers.
Mutation	An operation that changes stored or cached data.
Promise	A value representing eventual completion or failure.
Props	Inputs passed to a component by its parent.
Query key	The identifier of fetched data in the Query cache.
Responsive UI	A layout that adapts to screen size and input conditions.
RPC	Calling a named remote operation with arguments.
Runtime	The environment that executes the program.
Schema	A defined structure and constraints for data.
State	A value that describes the current situation and may change.
Validation	Checking that data has an acceptable shape and values.

43 / Glossary: security, data and intelligence

Term	Meaning in plain language
ACID	Atomicity, consistency, isolation and durability for transactions.
Audit trail	A record of who performed which important operation and when.
Authentication	Establishing the caller's identity.
Authorization	Deciding which actions and records that identity may access.
Calibration	Whether stated probabilities agree with observed outcome rates.
Cohort holdout	Keeping one time/group cohort separate for evaluation.
CORS	Browser rules for access to cross-origin responses.
CSRF	A forged action made through another person's signed-in browser.
Eligibility gate	A required condition checked before ranking or participation.
Epoch	One training pass through the available training rows.
Hash / salt	One-way representation / per-secret randomness for password storage.
HTTP-only	Cookie protection that prevents ordinary page scripts from reading it.
IDOR	Missing record ownership checks that let changed IDs expose other users' data.
Idempotency	Repeating an operation does not repeat its business effect.
Index	A database structure that accelerates suitable lookups.
Leakage	Training/evaluation contamination or using information unavailable before the outcome.
Logistic regression	A model producing a sigmoid-transformed linear score.
NLP	Processing human-language text into useful structure or measurements.
OAuth / PKCE	Delegated authorization flow / protection for the code exchange.
Outbox	Persistent pending-delivery records for later processing.
Overfitting	Learning details that do not generalize beyond training data.
Parameterized SQL	Queries that bind data separately from executable SQL.
Provenance	Where data or a model came from, such as synthetic or historical rows.
RBAC	Access decisions based on assigned roles.
RLS	Database policies restricting rows according to defined conditions.
TF-IDF / cosine	Term weighting / comparison of vector directions for relevance.
Transaction	A guarded group of database operations that commits or rolls back.

44 / Revision plan and project demonstration

Session 1: explain the structure

Practice the 30-second introduction and the architecture diagram. Define React, Next.js, Node, Express and the database. Explain why using the word "MERN" would misidentify this database layer. Find the relevant folders without opening private environment files.

Session 2: understand the interface

Explain props, state and effects using the institute editor. Describe the difference between React state, Zustand and Query. Show how the sidebar expands and how the phone layout changes. Name the tools used for forms, icons, charts and animations.

Session 3: trace a protected change

Walk through an institute record update from screen to transaction. Explain validation, authentication, authorization and auditing separately. Demonstrate that a successful edit changes the displayed record, and describe the tests that reject another institute's student and protected fields.

Session 4: explain progression

Complete a contest and inspect points, history, current streak and badge progress. Explain exactly-once awards, one streak day per calendar day, and the difference between a current streak and a longest historical streak. Mention the need for enough published contests for longer streaks.

Session 5: explain intelligence honestly

Separate eligibility, readiness, lexical relevance, trained prediction and generated advice. Write the hybrid score and logistic sigmoid formulas. Describe the held-out cohort and explain why synthetic evaluation cannot establish a real accuracy claim.

Session 6: practice communication and reliability

Show typed communication practice and the correction examples. Explain the browser recognition path and its boundaries before demonstrating a microphone. Describe AI consent, local fallback, private files, email jobs and hosted configuration. Answer the three interview-question sections aloud.

A compact presentation outline

- Problem, intended users and the three roles.
- Architecture and technology choices.
- One protected workflow with a visible result.
- Preparation features, voice practice and contest achievements.
- Testing evidence, current limitations and planned improvements.

Final speaking rule

Adapt the first-person answers to what you personally understand and contributed. If you have not configured or verified an external provider, say the integration is available in the code and describe its setup requirements. Knowing a limitation clearly is more credible than claiming every installed technology is fully deployed.

45 / Source evidence and further study

Primary project evidence

This guide was prepared from the CampusLink workspace at C:/CampusLink on 6 October 2026. The technology inventory uses package.json and package-lock.json. The behavior descriptions prioritize source implementation over older frontend-only documentation.

Area	Main source files
UI / routes	src/app/layout.tsx; src/components/platform.tsx; src/components/app-shell.tsx; src/app/dashboard.css
State / transport	src/hooks/use-platform.ts; src/store/session.ts; src/services/api/client.ts; src/services/api/remote.ts
Institute editing	src/features/team.tsx; institute-student-editor.tsx; src/utis/student-records.ts; server/services.ts
Achievements	src/utis/contest-achievements.ts; src/components/contest-progress.tsx; src/features/assessments.tsx
Voice / feedback	src/hooks/use-voice-practice.ts; src/utis/voice-recognition.ts; communication.ts; src/features/communication.tsx
Server / security	server/app.ts; auth.ts; google.ts; workspace.ts; db.ts; services.ts
Intelligence	src/utis/scoring.ts; server/nlp.ts; ml.ts; generative.ts; ml-client.ts
Files / delivery	server/storage.ts; mail.ts; reminders.ts; index.ts
Workflow / checks	src/services/drive.domain.ts; src/utis/placement.ts; server/app.test.ts; *.test.ts
Setup / hosting	BACKEND-SETUP.md; DEPLOYMENT.md; next.config.mjs; render.yaml; vercel.json

46 / Official references for further study

These primary references were checked while preparing the guide. The project source determines the current implementation. The links below help you continue learning the underlying technologies; current documentation may cover features outside the exact versions locked in this project.

Official documentation

- [React: Quick Start](#) - components, JSX and state basics.
- [Next.js: Server and Client Components](#) - rendering boundaries; current documentation may describe features beyond this project's locked Next 15 version.
- [TypeScript: Everyday Types](#) - type-system basics.
- [TanStack Query: Important Defaults](#) - cache and freshness behavior.
- [Express: Using Middleware](#) - request-processing order.
- [Tailwind: PostCSS Installation](#) - CSS build integration.
- [Zod: Basic Usage](#) - parsing and validation; this project uses the locked Zod 3 family.
- [Node 24: SQLite](#) - built-in database API; exact status is version-specific.
- [Node: Crypto](#) - hashing, scrypt and timing-safe comparison; project runtime is Node 24.
- [PostgreSQL: Transactions](#) - commit and rollback fundamentals.
- [Supabase: Google Sign-In](#) - provider setup.
- [OpenAI: Text Generation](#) and [Structured Outputs](#) - optional coaching API concepts.
- [MDN: SpeechRecognition](#) - browser recognition and availability caveats.

The explanations and interview answers are original project-oriented study notes. The linked documentation is for checking details and continuing your learning. Provider plans, availability and APIs can change; consult the linked primary documentation when configuring a live system.